

Hiero Shared SDK Automations

Five-Page Architecture Plan

arc42-compliant | Microkernel pattern | Draft for Sophie / explorer3 review | May 2026

Core principle: centralise reusable automation decision logic, not repository control.

Architecture pattern: Microkernel Architecture (Richards, 2015). The core automation kernel resides in `packages/core`; GitHub Actions and Probot are delivery adapters.

arc42 coverage: Sections 1-12 are addressed in this five-page plan. C4 views are included for L1 (Context), L2 (Container), and L3 (Component).

This document is intentionally architecture-first and follows the arc42 structure. C4 L1 is shown as a diagram; L2 and L3 are text views. A separate migration strategy follows from this architecture and is intentionally out of scope here.

1 Introduction and Goals

Hiero Shared SDK Automations provides a shared automation decision core while preserving SDK repository autonomy over workflow triggers, permissions, and security controls. The audience is SDK maintainers, automation contributors, and reviewers who need a concise, reviewable baseline.

1.1 Top goals (quality drivers)

- Security: shared logic must not hide permissions, tokens, or runner hardening.
- Maintainability: fix automation logic once and adopt deliberately across SDKs.
- Evolvability: enable Actions now and Probot later without duplicating behavior.
- Testability: prove parity before write mode through fixtures and comparisons.
- Adoptability: each SDK adopts incrementally and can roll back by SHA.

1.2 Stakeholders and expectations

Stakeholder	Expectation
SDK maintainers	Keep governance and policy control, approve rollout timing, and retain security gates.
SDK repositories	Keep workflow YAML, permissions, checkout, concurrency, harden-runner, and local config.
<code>sdk-automations</code>	Provide shared core logic, adapters, schemas, releases, fixtures, and examples.
GitHub Actions	Execute SDK-owned workflows that call central actions by pinned references.
Future GitHub App / Probot	Later orchestration path that reuses the same core without a second implementation.

2 Constraints

- GitHub Actions is the current execution environment; workflows remain in SDK repos.
- SDK repos retain security controls: permissions, harden-runner, checkout, concurrency.
- Central repo must be org-owned and governed before production dependency.
- Repo policy config is protected and must remain local to each SDK.
- No privileged tokens or escalated permissions originate from the shared core.
- Cross-SDK behavior differences require parity tests before write mode.

3 Context and Scope

The architecture boundary is deliberate: SDK repositories remain the security and governance boundary, while `sdk-automations` becomes the shared implementation boundary for reusable automation logic.

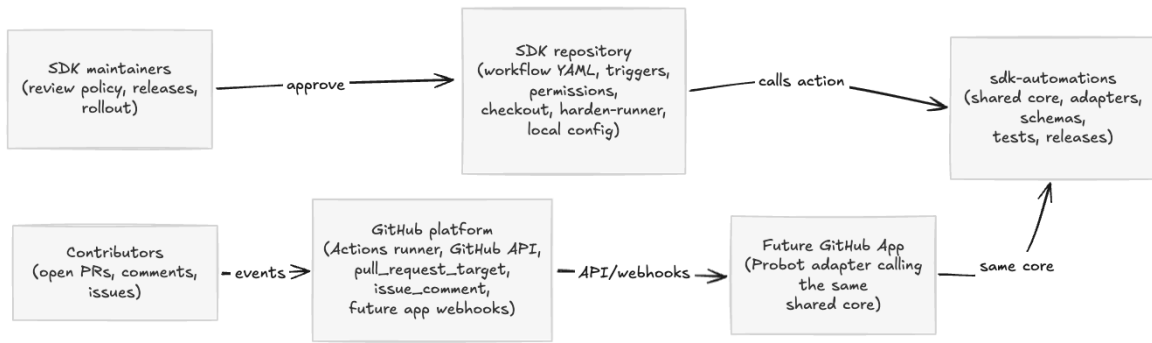


Figure 1: C4 L1 context view for shared SDK automations (Excalidraw).

Phase 0 proof: a [Python fork canary run](#) validated the end-to-end call path while keeping workflow boundaries local and applying expected review labels.

3.1 Scope

In scope:

- Shared automation decision logic, schemas, fixtures, and adapters.
- SDK-owned workflows and protected repo policy configuration.
- Incremental adoption by SDKs with rollback by pinned SHA.

Out of scope: removing all SDK workflows, bypassing maintainer approval, moving privileged policy into a personal fork, or forcing C++ workflows onto an unproven wrapper.

4 Solution Strategy

Adopt a shared-core with ports-and-adapters architecture aligned to the Microkernel pattern. The core houses reusable automation decisions, while adapters map runtime inputs from GitHub Actions and (later) Probot to the same core API. GitHub events trigger the system, but the architecture is not a hosted microservice first; it is a modular automation kernel with thin delivery adapters.

4.1 Microkernel pattern fit

Attribute	Expected fit	Rationale for this system
Agility	High	New or changing automations are plug-ins in the core without rewriting runtimes.
Testability	High	Core logic is isolated and verified with fixtures and parity tests.
Deployability	High	Adapters and actions are versioned and pinned independently per SDK.
Scalability	Medium-Low	Workload is bounded by SDK events, not high-throughput service traffic.
Simplicity	Medium	Clear core/adapter boundaries reduce duplication but require adapter discipline.

5 Building Block View

5.1 C4 L2 - Container view (text)

Container	Responsibility and interaction
SDK repositories	Own workflows, policy config, triggers, and permissions; call shared actions.
sdk-automations repo	Hosts core logic, adapters, schemas, actions, fixtures, and releases.
GitHub Actions runtime	Executes SDK workflows and runs shared actions.
Future GitHub App / Probot	Alternative runtime that calls the same core API.
GitHub API	External system for comments, labels, assignments, and PR metadata.

5.2 C4 L3 - Component view (text)

Component	Role
packages/core	Automation decision kernel, pure logic, no runtime-specific I/O.
packages/github-action-adapter	Validates inputs, loads config, maps Action runtime to core calls.
packages/probot-app	Maps webhook events to the same core interface used by Actions.
actions/review-sync	Thin action entrypoint that calls the Action adapter.
schemas and fixtures	Shared config schema, validation, and parity test fixtures.

5.3 Three-bucket classification

Bucket	Examples	Rule
Shared core candidate	Review-sync decisions, assignment eligibility, PR check rules, comment builders, label transition decisions, fixtures, parity tests, PR priority scoring (issue label -> weight + age factor).	Move here when behavior is common or intentionally configurable across SDKs.
Repo-local boundary	Workflow triggers, permissions, checkout, harden-runner, concurrency, artifact download / upload, token choice.	Keep local because these are security and operational controls.
Repo-specific policy	Maintainer team, label names, documentation links, assignment limits, support channels.	Keep as protected local config validated by the central schema.

5.4 Candidate repository shape

The candidate central repository should expose the following components:

- packages/core
- packages/github-action-adapter
- packages/probot-app
- actions/review-sync
- actions/run-automation
- schemas
- docs
- examples
- CI

Personal fork links are proof artifacts only; upstream SDKs should depend only on an approved org-owned repository with maintainers and release governance.

6 Runtime View

6.1 GitHub Actions runtime

1. The SDK workflow starts from its own trigger, permissions, concurrency, and maintainer-approved YAML.

2. The job keeps **step-security/harden-runner** visible near the start of the job and checks out only what the job is allowed to use.
3. The workflow loads protected repo config from the trusted branch and calls the shared action by a reviewed immutable reference.
4. The Action adapter validates inputs and config, then calls **packages/core** for the automation decision.
5. The core returns dry-run output or performs allowed writes through the GitHub API, depending on the approved workflow mode.

6.2 Probot runtime

1. A future GitHub App receives the webhook and authenticates as the app installation.
2. The Probot adapter maps the event into the same core function shape used by Actions.
3. The core produces the same comments, labels, assignment decisions, and safety checks.

7 Deployment View

- Production callers should pin central actions and third-party actions to full-length commit SHAs, using tags only as human-readable release labels.
- The central repo must move under the approved official owner before upstream SDK production dependency.
- Branch protection, CODEOWNERS, release owners, changelog, rollback instructions, and incident ownership are part of initial governance, not optional extras.
- Repo config is privileged policy and must require protected-branch review because changing config changes automation behavior.

Migration sequence implied by this architecture:

- Phase 0 (complete): Python fork canary proves the caller contract.
- Phase 1: Stabilize central repo governance before any upstream dependency.
- Phase 2: Python review-sync dry-run pilot upstream (no existing workflow replaced).
- Phase 3: Controlled write pilot with named owner and rollback SLA.
- Phase 4+: Assignment mapping, then C++ canary, then wider SDK rollout.

8 Crosscutting Concepts

- **Security:** workflow YAML remains local, permissions are explicit, and harden-runner stays visible in SDK workflows.
- **Configuration:** per-SDK policy config is validated by shared schemas.
- **Testing:** fixtures and parity tests validate core decisions against existing behavior.
- **Observability:** consistent logs and dry-run outputs for reviewers and maintainers.
- **Versioning:** SDKs pin SHAs and can roll back by reverting a single caller change.

9 Architectural Decisions

- **Microkernel pattern:** the shared core is the automation kernel, adapters are plug-ins.
- **Actions-first, Probot-ready:** use GitHub Actions for early pilots; the ports-and-adapters split keeps **packages/core** stable as runtimes change, so a Probot adapter maps webhook events to the same function signatures without core rewrites.
- **Do not centralise workflow YAML:** centralise logic only where duplication creates real maintenance cost.
- **Python review-sync first:** start with lower-risk automation already proven in a shared-action form.

- **Assignment logic deferred:** wait for a Python/C++ behavior map before extraction.

The migration strategy is documented separately and derived from this architecture.

10 Quality Requirements

Quality goal	Scenario or test
Security	A contributor-facing workflow can call the shared action without hiding harden-runner, expanding token permissions unexpectedly, or checking out untrusted PR code in a privileged job.
Maintainability	A bug fix in shared review-sync or assignment logic is tested once in <code>packages/core</code> and released once for SDK repos to adopt deliberately.
Evolvability	A future Probot / GitHub App adapter can call the same core functions without a second implementation of assignment or PR checks.
Testability	Before write mode, golden fixtures and behavior parity tests prove equivalence with existing SDK behavior, not only human dry-run log review.
Adoptability	Each SDK can adopt one automation at a time, keep local policy, and roll back by reverting one caller change or pinning an older approved SHA.

11 Risks and Technical Debt

Risk or debt	Mitigation or note
Shared core defect impacts multiple SDKs	Pin versions, stage rollout, and roll back by SHA; keep parity tests green.
Behavior drift between SDKs	Maintain behavior maps, per-SDK fixtures, and strict schema validation.
Central repo governance delayed	Keep local automations active and treat shared core as internal until approved.
Security regression from hidden permissions	Keep workflow YAML local and require security review for any permission change.
C++ wrapper constraints unknown	Keep C++ workflows local until a proven wrapper and parity results exist.

12 Glossary

Term	Meaning
<code>sdk-automations</code>	Central repository hosting shared automation logic and adapters.
<code>packages/core</code>	Automation decision kernel, runtime-agnostic logic.
Adapter	Thin layer that maps runtime inputs to the core interface.
<code>harden-runner</code>	StepSecurity action that hardens GitHub Actions runners.
dry-run mode	Non-writing execution that outputs intended actions for review.
parity tests	Tests that compare core outputs against existing SDK behavior.
<code>review-sync</code>	Automation that synchronizes review state, labels, or comments.

References

- arc42 overview: <https://arc42.org/overview>
- Architecture documentation guide and C4 framing: <https://www.workingsoftware.dev/software-architecture-documentation-the-ultimate-guide/>
- Richards, Mark. *Software Architecture Patterns*. O'Reilly, 2015.
- GitHub Actions secure use reference: <https://docs.github.com/en/actions/reference/security/secure-use>
- StepSecurity Harden-Runner docs: <https://docs.stepsecurity.io/harden-runner>